

LES BASES DU DÉVELOPPEMENT WEB

Comprendre le Web, le HTML, le CSS, le JavaScript
et l'architecture client-serveur

Cours d'introduction — Développement web front-end & back-end
Support pédagogique

Sommaire

1. Introduction au développement web
2. Comment fonctionne le Web ? (client-serveur, HTTP, DNS)
3. HTML — la structure des pages
4. CSS — la mise en forme
5. JavaScript — l'interactivité
6. Front-end vs Back-end
7. Les bases de données
8. Outils et environnement de travail
9. Bonnes pratiques du développeur web
10. Exercices et projet fil rouge
11. Glossaire

1. Introduction au développement web

Le développement web désigne l'ensemble des techniques permettant de concevoir, construire et maintenir des sites et applications accessibles via un navigateur (Chrome, Firefox, Edge, Safari...). C'est un domaine vaste qui combine des langages de programmation, des protocoles réseau, des bases de données et des principes de design d'interface.

1.1 Deux grandes familles de compétences

- **Le Front-end** : tout ce que l'utilisateur voit et manipule directement dans son navigateur (mise en page, couleurs, boutons, animations).
- **Le Back-end** : la partie invisible, exécutée sur le serveur (traitement des données, logique métier, sécurité, accès à la base de données).

■ À retenir

Un développeur capable de travailler sur les deux facettes (front-end et back-end) est appelé **développeur full-stack**. C'est souvent le profil visé en début de formation.

1.2 Pourquoi apprendre le développement web ?

Le Web est aujourd'hui le principal canal de diffusion des services numériques : e-commerce, réseaux sociaux, plateformes d'apprentissage en ligne, applications bancaires, etc. Savoir développer pour le Web permet de créer des outils accessibles à n'importe qui, sur n'importe quel appareil, sans installation préalable.

2. Comment fonctionne le Web ?

Avant d'écrire la moindre ligne de code, il est essentiel de comprendre ce qui se passe lorsqu'un utilisateur tape une adresse dans son navigateur.

2.1 Le modèle client-serveur

Le Web repose sur une architecture **client-serveur** : le **client** (le navigateur) envoie une requête, et le **serveur** (un ordinateur distant qui héberge le site) renvoie une réponse.

- 1 L'utilisateur saisit une URL (ex. `www.exemple.com`) dans son navigateur.
- 2 Le navigateur interroge un serveur DNS pour convertir le nom de domaine en adresse IP.
- 3 Le navigateur envoie une requête HTTP au serveur correspondant à cette adresse IP.
- 4 Le serveur traite la requête (recherche de fichiers, calculs, accès base de données...).
- 5 Le serveur renvoie une réponse HTTP contenant le code HTML/CSS/JS de la page.
- 6 Le navigateur interprète ce code et affiche la page à l'écran.

2.2 Le protocole HTTP

HTTP (HyperText Transfer Protocol) est le langage utilisé par le client et le serveur pour communiquer. Chaque requête possède une **méthode** qui indique l'action souhaitée :

Méthode	Rôle	Exemple d'usage
GET	Récupérer une ressource	Afficher une page ou une liste de cours
POST	Envoyer des données pour créer une ressource	Soumettre un formulaire d'inscription
PUT / PATCH	Modifier une ressource existante	Mettre à jour un profil étudiant
DELETE	Supprimer une ressource	Supprimer une leçon ou un compte

Chaque réponse contient également un **code de statut** qui indique le résultat de la requête :

Code	Signification
200 OK	La requête a réussi
301 / 302	Redirection vers une autre adresse
404 Not Found	Ressource introuvable
403 Forbidden	Accès refusé
500 Internal Server Error	Erreur côté serveur

2.3 HTTPS et sécurité

HTTPS est la version sécurisée de HTTP : les données échangées entre le client et le serveur sont chiffrées grâce au protocole TLS, ce qui empêche un tiers d'intercepter des informations sensibles (mots de passe, données bancaires...).

3. HTML — la structure des pages

HTML (HyperText Markup Language) n'est pas un langage de programmation mais un **langage de balisage**. Il sert à structurer le contenu d'une page : titres, paragraphes, images, liens, formulaires...

3.1 Structure minimale d'un document HTML

```
<!DOCTYPE html>
<html lang="fr">
<head>
<meta charset="UTF-8">
<title>Ma première page</title>
</head>
<body>
<h1>Bienvenue sur mon site</h1>
<p>Ceci est un paragraphe d'introduction.</p>
</body>
</html>
```

<!DOCTYPE html> indique au navigateur qu'il s'agit d'un document HTML5. La balise **<head>** contient des informations invisibles (titre de l'onglet, encodage, liens vers les fichiers CSS). La balise **<body>** contient tout le contenu visible de la page.

3.2 Les balises essentielles

Balise	Usage
<h1> à <h6>	Titres, du plus important au moins important
<p>	Paragraphe de texte
	Lien hypertexte
	Image
/ /	Liste à puces / numérotée / élément de liste
<div>	Conteneur générique en bloc
	Conteneur générique en ligne
<table>, <tr>, <td>	Tableau, ligne, cellule
<form>, <input>, <button>	Formulaire et champs de saisie

3.3 Exemple de formulaire

```
<form action="traitement.php" method="POST">
<label for="nom">Nom :</label>
<input type="text" id="nom" name="nom" required>

<label for="email">Email :</label>
<input type="email" id="email" name="email" required>

<button type="submit">Envoyer</button>
</form>
```

■ Bonne pratique

L'attribut **method="POST"** est recommandé pour l'envoi de données sensibles ou destinées à être enregistrées, car elles ne sont pas visibles dans l'URL, contrairement à la méthode GET.

4. CSS — la mise en forme

CSS (Cascading Style Sheets) permet de définir l'apparence visuelle des éléments HTML : couleurs, polices, espacements, mise en page, animations...

4.1 Trois façons d'ajouter du CSS

- **CSS externe** (recommandé) : un fichier .css séparé, lié via `<link rel="stylesheet" href="style.css">`.
- **CSS interne** : à l'intérieur d'une balise `<style>` dans le `<head>`.
- **CSS en ligne** : directement dans l'attribut `style="..."` d'une balise (à éviter, difficile à maintenir).

4.2 Syntaxe de base

```
/* Sélecteur { propriété: valeur; } */
h1 {
  color: #6C3FC5;
  font-size: 28px;
  text-align: center;
}

.carte {
  background-color: #ffffff;
  border-radius: 8px;
  padding: 16px;
  box-shadow: 0 2px 6px rgba(0,0,0,0.1);
}

#menu-principal {
  display: flex;
  justify-content: space-between;
}
```

Un **sélecteur de balise** (h1) cible toutes les balises du même type. Un **sélecteur de classe** (.carte) cible tous les éléments possédant `class="carte"`. Un **sélecteur d'identifiant** (#menu-principal) cible un élément unique possédant `id="menu-principal"`.

4.3 Le modèle de boîte (Box Model)

Chaque élément HTML est traité comme une boîte rectangulaire composée de quatre couches, de l'intérieur vers l'extérieur :

- 1 **content** : le contenu lui-même (texte, image...).
- 2 **padding** : l'espace intérieur entre le contenu et la bordure.
- 3 **border** : la bordure de la boîte.
- 4 **margin** : l'espace extérieur entre la boîte et ses voisines.

4.4 Mise en page moderne : Flexbox et Grid

Flexbox et CSS Grid sont deux systèmes de mise en page qui ont remplacé les anciennes techniques basées sur les flottants (float). Flexbox est idéal pour aligner des éléments sur un axe (ligne ou colonne), tandis que Grid permet de construire des mises en page en deux dimensions (lignes et

colonnes).

```
.conteneur {  
  display: flex;  
  gap: 16px;  
  flex-wrap: wrap;  
}
```


5. JavaScript — l'interactivité

JavaScript (JS) est le langage de programmation qui rend une page web **interactive** : validation de formulaires, animations, mise à jour du contenu sans recharger la page, communication avec un serveur...

5.1 Insérer du JavaScript

```
<!-- Dans le HTML -->
<script src="script.js"></script>

<!-- Ou directement -->
<script>
console.log("Page chargée !");
</script>
```

5.2 Manipuler le DOM

Le DOM (Document Object Model) est la représentation en mémoire de la page HTML. JavaScript permet de lire et modifier cette structure en temps réel, ce qui change ce que voit l'utilisateur sans recharger la page.

```
// Sélectionner un élément
const titre = document.querySelector("h1");

// Modifier son contenu
titre.textContent = "Nouveau titre";

// Réagir à un clic
document.querySelector("#btn-valider").addEventListener("click", () => {
  alert("Formulaire validé !");
});
```

5.3 Communiquer avec un serveur : fetch et AJAX

La technique AJAX permet d'envoyer et de recevoir des données depuis un serveur sans recharger la page. En JavaScript moderne, on utilise généralement la fonction **fetch**.

```
fetch("api/etudiants.php")
  .then(reponse => reponse.json())
  .then(donnees => {
    console.log(donnees);
  })
  .catch(erreur => console.error("Erreur :", erreur));
```

■ Le saviez-vous ?

C'est exactement ce principe (AJAX + PHP) qui permet de créer des interfaces dynamiques comme un tableau de bord d'étudiant qui se met à jour sans rechargement, ou un jeu multijoueur en ligne basé sur un système de "polling" (interrogation régulière du serveur).

6. Front-end vs Back-end

Une application web complète combine généralement une partie front-end et une partie back-end, qui communiquent entre elles via des requêtes HTTP.

Aspect	Front-end	Back-end
Exécution	Dans le navigateur du client	Sur le serveur
Langages courants	HTML, CSS, JavaScript	PHP, Python, Java, Node.js...
Rôle	Affichage, interaction utilisateur	Logique métier, sécurité, données
Exemples de tâches	Mise en page, formulaires, animations	Authentification, calculs, requêtes SQL
Visible par l'utilisateur ?	Oui (code source consultable)	Non (code exécuté côté serveur)

6.1 Exemple concret : une page de connexion

- 1 Le **front-end** affiche un formulaire HTML avec des champs identifiant/mot de passe.
- 2 JavaScript peut vérifier que les champs ne sont pas vides avant l'envoi.
- 3 Les données sont envoyées via une requête POST vers le **back-end** (par exemple un script PHP).
- 4 Le back-end vérifie les identifiants dans la **base de données**.
- 5 Le serveur renvoie une réponse (succès ou erreur), que le front-end affiche à l'utilisateur.

6.2 Le rôle d'une API

Une API (Application Programming Interface) est un ensemble de règles qui permet à deux logiciels de communiquer. Sur le Web, on parle souvent d'API REST : le front-end envoie des requêtes HTTP à des "routes" précises (ex. /api/cours, /api/etudiants) et reçoit des données structurées, généralement au format JSON.

```
{
  "id": 12,
  "nom": "Introduction au HTML",
  "duree_minutes": 45,
  "termine": false
}
```

7. Les bases de données

Une base de données permet de stocker, organiser et retrouver efficacement de grandes quantités d'informations : utilisateurs, articles, commandes, notes, etc.

7.1 Bases de données relationnelles

Les bases de données relationnelles (comme MySQL, utilisée avec PHP dans l'environnement XAMPP) organisent les données en **tables** composées de lignes et de colonnes, reliées entre elles par des clés.

Concept	Définition
Table	Ensemble structuré de données (ex. table "utilisateurs")
Colonne / Champ	Une caractéristique stockée (ex. nom, email)
Ligne / Enregistrement	Une entrée précise dans la table (ex. un utilisateur donné)
Clé primaire	Identifiant unique de chaque ligne (souvent "id")
Clé étrangère	Référence vers la clé primaire d'une autre table, pour créer une relation

7.2 Le langage SQL

SQL (Structured Query Language) est le langage standard pour interroger une base de données relationnelle.

```
-- Créer une table
CREATE TABLE utilisateurs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  mot_de_passe VARCHAR(255) NOT NULL
);

-- Insérer une ligne
INSERT INTO utilisateurs (nom, email, mot_de_passe)
VALUES ('Awono', 'awono@example.com', 'motdepasse_hache');

-- Sélectionner des données
SELECT nom, email FROM utilisateurs WHERE id = 1;
```

■ Sécurité

Ne jamais insérer directement les données saisies par l'utilisateur dans une requête SQL (risque d'**injection SQL**). Il faut utiliser des **requêtes préparées** (prepared statements), disponibles en PHP via PDO ou MySQLi.

7.3 Autres types de bases de données

- **Bases NoSQL** (ex. MongoDB) : stockent des données sous forme de documents flexibles, utiles pour des données peu structurées.
- **Bases légères** (ex. SQLite) : un fichier unique, sans serveur dédié, pratique pour de petits projets ou des prototypes.

8. Outils et environnement de travail

8.1 Le navigateur et ses outils de développement

Tous les navigateurs modernes intègrent des outils de développement (touche F12) permettant d'inspecter le HTML/CSS, de déboguer le JavaScript via la console, et d'analyser les requêtes réseau.

8.2 Un éditeur de code

Un éditeur comme Visual Studio Code facilite l'écriture du code grâce à la coloration syntaxique, l'auto-complétion et de nombreuses extensions (Live Server, PHP Intelephense, Prettier...).

8.3 Un serveur local : XAMPP

XAMPP est une distribution qui regroupe Apache (serveur web), MySQL/MariaDB (base de données), et PHP sur un même poste. Elle permet de développer et tester une application web en conditions proches d'un vrai serveur, sans être connecté à Internet.

Composant	Rôle
Apache	Serveur web qui reçoit les requêtes HTTP et exécute les scripts PHP
MySQL / MariaDB	Système de gestion de base de données relationnelle
PHP	Langage back-end exécuté côté serveur
phpMyAdmin	Interface web pour gérer visuellement la base de données

8.4 Le contrôle de version : Git

Git permet de suivre l'historique des modifications d'un projet et de collaborer à plusieurs. GitHub est une plateforme en ligne pour héberger des dépôts Git et les partager.

```
git init
git add .
git commit -m "Premier commit"
git remote add origin https://github.com/utilisateur/projet.git
git push -u origin main
```

9. Bonnes pratiques du développeur web

- **Séparer les responsabilités** : structure (HTML), présentation (CSS) et comportement (JavaScript) dans des fichiers distincts.
- **Nommer clairement** les fichiers, variables et fonctions pour faciliter la relecture.
- **Commenter le code** lorsque la logique n'est pas évidente, sans excès.
- **Valider et sécuriser** toutes les données envoyées par un utilisateur (formulaires, URL...).
- **Tester régulièrement** sur différents navigateurs et tailles d'écran (responsive design).
- **Utiliser un système de versionnage** (Git) dès le début d'un projet, même en solo.
- **Éviter la duplication de code** en créant des fonctions ou des composants réutilisables.

■ Accessibilité

Penser à l'accessibilité (attribut alt sur les images, contrastes suffisants, navigation au clavier) rend un site utilisable par le plus grand nombre, y compris les personnes en situation de handicap.

10. Exercices et projet fil rouge

10.1 Exercices de compréhension

- 1 Expliquez avec vos propres mots la différence entre une requête GET et une requête POST.
- 2 Quelle est la différence entre une balise <div> et une balise ?
- 3 Écrivez le code CSS pour qu'un titre <h1> soit centré, en violet, avec une taille de police de 32px.
- 4 Pourquoi est-il déconseillé d'utiliser du CSS en ligne (attribut style) pour un projet de grande taille ?
- 5 Qu'est-ce qu'une clé primaire dans une base de données relationnelle ? Donnez un exemple.
- 6 Expliquez le principe d'une requête AJAX en une ou deux phrases.

10.2 Projet pratique

Réalisez une petite page « Carte de visite en ligne » comportant :

- Une structure HTML avec un titre, une photo, une courte présentation et une liste de compétences.
- Une feuille de style CSS externe utilisant Flexbox pour organiser la mise en page.
- Un bouton JavaScript qui affiche ou masque une section « Contact » au clic.
- (Bonus) Un formulaire de contact qui envoie les données vers un script PHP via la méthode POST.

11. Glossaire

Terme	Définition
API	Interface permettant à deux logiciels de communiquer entre eux
DOM	Représentation en mémoire de la structure HTML d'une page
Framework	Ensemble d'outils et de conventions facilitant le développement (ex. Laravel, React)
JSON	Format léger d'échange de données, lisible par l'humain et la machine
Requête HTTP	Message envoyé par un client à un serveur pour demander une ressource ou une action
Responsive design	Approche visant à adapter l'affichage d'un site à toutes les tailles d'écran
Serveur	Ordinateur distant qui héberge et sert les ressources d'un site web
SGBD	Système de Gestion de Base de Données (ex. MySQL, PostgreSQL)
URL	Adresse unique permettant de localiser une ressource sur le Web

Fin du support — bon apprentissage et bon code !